Report

Shortest Route Between Railway Stations using Dijkstra's Algorithm

1. Problem Statement

Develop a C++ program to represent railway stations as vertices and track links as weighted edges, where weights represent distance or travel cost. Apply Dijkstra's algorithm from a selected station and display the shortest distance and station sequence to a selected destination.

2. Objective

To represent a railway network as a weighted graph, with stations as vertices and tracks as edges, and apply Dijkstra's algorithm to find the shortest distance and station sequence between a selected source and destination, handling invalid stations and unreachable routes.

3. Algorithm

Step 1: START.

Step 2: Build the graph as an adjacency list: each station is a vertex and each track is an undirected edge (u, v) with a non-negative weight w (distance in km).

Step 3: Read the source and destination stations. If either station is not present in the graph, display an error and STOP.

Step 4: For every station, initialize dist[] = INFINITY, parent[] = -1 and visited[] = false.

Step 5: Set dist[source] = 0 and push (0, source) into a min-priority queue.

Step 6: WHILE the priority queue is not empty, repeat Steps 7 to 9. When it becomes empty, go to Step 10.

Step 7: Remove the pair (d, u) with the smallest distance from the queue. If visited[u] = true, skip it and go back to Step 6.

Step 8: Mark visited[u] = true.

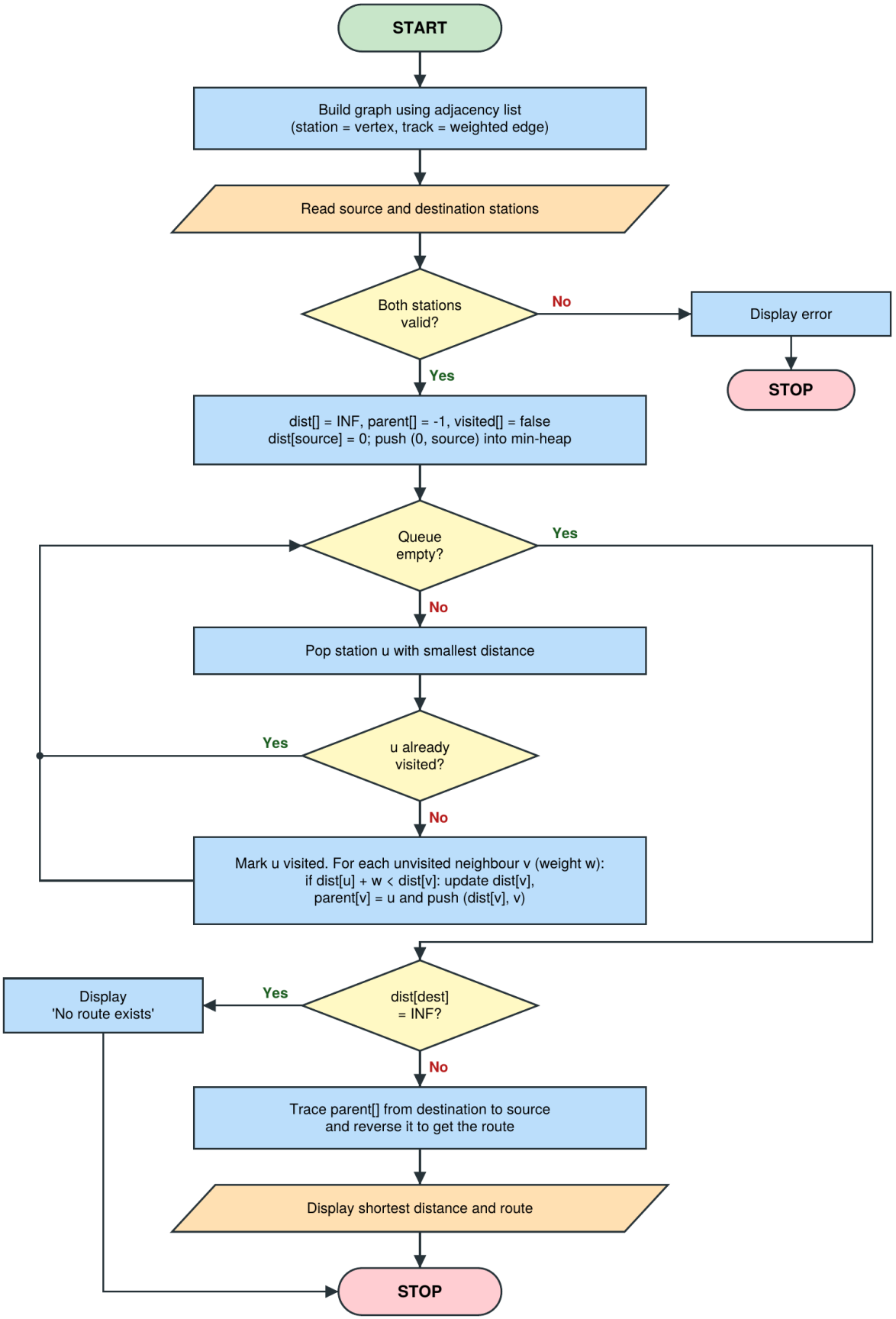
Step 9: For each unvisited neighbour v of u with edge weight w: if dist[u] + w < dist[v], then set dist[v] = dist[u] + w and parent[v] = u, and push (dist[v], v) into the queue.

Step 10: If dist[destination] = INFINITY, display "No route exists" and STOP.

Step 11: Build the route: starting from the destination, follow parent[] until the source is reached (parent = -1), then reverse the sequence.

Step 12: Display dist[destination] as the shortest distance and the reversed sequence as the station route. STOP.

4. Flowchart


START
Build graph using adjacency list
(station = vertex, track = weighted edge)
Read source and destination stations
Both stations valid?
No
Display error
STOP
Yes
dist[] = INF, parent[] = -1, visited[] = false
dist[source] = 0; push (0, source) into min-heap
Queue empty?
Yes
No
Pop station u with smallest distance
u already visited?
Yes
No
Mark u visited. For each unvisited neighbour v (weight w):
if dist[u] + w < dist[v]: update dist[v],
parent[v] = u and push (dist[v], v)
dist[dest] = INF?
Yes
Display 'No route exists'
No
Trace parent[] from destination to source
and reverse it to get the route
Display shortest distance and route
STOP